

Ayudantía N°4

Felipe Llach R.

07-05-2026

Ejercicio 1: Patrón Singleton (El Vigilante de la Configuración)

Contexto:

Estás desarrollando un sistema de seguridad para una bóveda bancaria. El sistema tiene múltiples terminales de acceso, pero todas deben consultar las mismas reglas de seguridad (como el horario de apertura y el nivel de alerta actual). Si cada terminal creara su propio objeto de configuración, los datos podrían desincronizarse y causar un fallo crítico de seguridad. **Requerimientos:**

- a) Crear una clase llamada `ConfiguracionSeguridad`.
- b) Implementar el patrón **Singleton** para asegurar que exista una única instancia global.
- c) El constructor debe ser privado para evitar instanciación externa.
- d) Definir una propiedad `NivelAlerta` (string).
- e) En el método `Main`, demostrar que al modificar la propiedad desde una variable `terminalA`, el cambio es visible inmediatamente al consultar una variable `terminalB`.

Ejercicio 2: Patrón Composite (Estructura Organizacional)

Contexto:

Una empresa de tecnología requiere una herramienta para visualizar su organigrama. La empresa está compuesta por empleados individuales (como Programadores o Diseñadores), pero también por Departamentos que pueden contener tanto empleados como otros sub-departamentos. **Requerimientos:**

- a) Definir una interfaz `IComponenteEmpleado` con el método `MostrarDetalles()`.

- b) Implementar la clase **Empleado** (Hoja/Leaf) que represente a un trabajador individual.
- c) Implementar la clase **Departamento** (Compuesto/Composite) que contenga una lista de **IComponenteEmpleado**.
- d) El método **MostrarDetalles()** del departamento debe ser recursivo: debe imprimir el nombre del departamento y luego llamar al mismo método de todos sus elementos hijos.

Ejercicio 3: Integración Singleton + Composite (Motor de GUI)

Contexto:

Se solicita diseñar un motor de renderizado de interfaces gráficas. Los elementos de la interfaz (Botones y Paneles) siguen una estructura jerárquica, pero el lienzo de dibujo (Canvas) debe ser único para evitar conflictos de hardware. **Requerimientos:**

- a) **Parte Composite:** Implementar una jerarquía donde un **Panel** pueda contener múltiples **Botones** u otros **Paneles**. Todos deben compartir un método **Dibujar()**.
- b) **Parte Singleton:** Crear una clase **RenderizadorCanvas** como Singleton. Esta clase será la encargada de gestionar el acceso exclusivo a la pantalla.
- c) **Integración:** Cada vez que se llame al método **Dibujar()** de un componente, este debe solicitar la instancia única del **RenderizadorCanvas** para imprimir un log de la operación (ej: "Dibujando elemento X en el Canvas único").